



Machine Learning (Praktikum)

SI – I₂

Imbalanced Data

Muhammad Felda Fahmi (187241080)
Faisal Akbar (187241107)

11 Mei 2026

Dosen Pengajar: PURBANDINI, S.Si., M.Kom.

Fakultas Sains dan Teknologi

Universitas Airlangga

2026

Praktikum

1. Tentang Dataset

- Dataset: Vehicle dataset (CAR DETAILS FROM CAR DEKHO.csv)
- Sumber Dataset:
<https://www.kaggle.com/datasets/nehalbirla/vehicle-dataset-from-cardekho>

2. Penjelasan Singkat Dataset

Dataset "Car details v3.csv" adalah kumpulan data yang berisi informasi mengenai spesifikasi teknis dan harga mobil bekas. Dataset ini sangat cocok digunakan untuk membangun model *Machine Learning* yang memprediksi harga jual mobil (*selling_price*) berdasarkan fitur-fiturnya.

a. Data memiliki 8.128 baris (jumlah mobil) dan memiliki 13 kolom/fitur.

b. Kondisi Data:

- Mayoritas data lengkap, namun terdapat nilai yang hilang (missing values) pada beberapa kolom spesifikasi teknis (sekitar 215 - 222 baris kosong) yaitu pada kolom mileage, engine, max_power, torque, dan seats.
- Terdapat beberapa kolom teknis (mileage, engine, max_power) yang masih menyimpan teks satuan di dalamnya (seperti "kmpl", "CC", "bhp"). Sesuai kode *preprocessing* yang Anda bagikan sebelumnya, kolom ini perlu dibersihkan untuk mengambil angka desimalnya saja agar bisa diproses oleh algoritma prediksi.

3. Penjelasan Code dan Hasil

Cell 1: Load Data dan Preprocessing

```
import pandas as pd
import numpy as np
import re
from datetime import datetime
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# 1. LOAD DATASET BARU SECARA EKSPLISIT
df = pd.read_csv('Car details v3.csv')

print("### Descriptive Statistics:")
print(df.describe())

print("\n### DataFrame Info:")
print(df.info())

print("\n### Missing values:")
print(df.isnull().sum())
```

```

... ### Descriptive Statistics:
      year  selling_price  km_driven  seats
count  8128.000000  8.128000e+03  8.128000e+03  7907.000000
mean    2013.804011  6.382718e+05  6.981951e+04  5.416719
std       4.044249  8.062534e+05  5.655055e+04  0.959588
min     1983.000000  2.999900e+04  1.000000e+00  2.000000
25%     2011.000000  2.549990e+05  3.500000e+04  5.000000
50%     2015.000000  4.500000e+05  6.000000e+04  5.000000
75%     2017.000000  6.750000e+05  9.800000e+04  5.000000
max     2020.000000  1.000000e+07  2.360457e+06  14.000000

### DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8128 entries, 0 to 8127
Data columns (total 13 columns):
#   Column      Non-Null Count  Dtype
---  -
0   name        8128 non-null   object
1   year        8128 non-null   int64
2   selling_price  8128 non-null   int64
3   km_driven    8128 non-null   int64
4   fuel        8128 non-null   object
5   seller_type  8128 non-null   object
6   transmission  8128 non-null   object
7   owner       8128 non-null   object
8   mileage     7907 non-null   object
9   engine      7907 non-null   object
10  max_power   7913 non-null   object
11  torque     7906 non-null   object
12  seats      7907 non-null   float64
dtypes: float64(1), int64(3), object(9)
memory usage: 825.6+ KB
None

### Missing values:
name        0
year        0
selling_price  0
km_driven    0
fuel        0
seller_type  0
transmission  0
owner       0
mileage     221
engine      221
max_power   215
torque     222
seats      221
dtype: int64

```

```

# 2. FEATURE ENGINEERING & CLEANING
df_processed = df.copy()

# Hapus baris kosong agar fungsi ekstraksi angka tidak error
df_processed.dropna(inplace=True)

# Fungsi untuk mengambil angka dari teks (misal: "1248 CC" -> 1248.0)
def extract_number(text):
    if pd.isnull(text): return np.nan
    res = re.findall(r"[-+]?[0-9]*\.?[0-9]+", str(text))
    return float(res[0]) if res else np.nan

# Bersihkan kolom spesifikasi teknis
df_processed['mileage'] = df_processed['mileage'].apply(extract_number)
df_processed['engine'] = df_processed['engine'].apply(extract_number)
df_processed['max_power'] = df_processed['max_power'].apply(extract_number)

# Ekstraksi Merek (kata pertama) & Usia Kendaraan
df_processed['brand'] = df_processed['name'].apply(lambda x: str(x).split(' ')[0])
df_processed['age'] = datetime.now().year - df_processed['year']

# Buang kolom yang sudah tidak terpakai atau kotor (torque di-drop karena formatnya sangat berantakan)
df_processed.drop(columns=['name', 'year', 'torque'], inplace=True)

print("### Descriptive Statistics:")
print(df_processed.describe())

print("\n### DataFrame Info:")
print(df_processed.info())

print("\n### Missing values:")
print(df_processed.isnull().sum())

```

```

... ### Descriptive Statistics:
      selling_price    km_driven    mileage    engine    max_power \
count  7.906000e+03  7.906000e+03  7906.000000  7906.000000  7906.000000
mean    6.498137e+05  6.918866e+04  19.419861  1458.708829  91.587374
std     8.135827e+05  5.679230e+04  4.036263   503.893057  35.747216
min     2.999900e+04  1.000000e+00  0.000000   624.000000  32.800000
25%     2.700000e+05  3.500000e+04  16.780000  1197.000000  68.050000
50%     4.500000e+05  6.000000e+04  19.300000  1248.000000  82.000000
75%     6.900000e+05  9.542500e+04  22.320000  1582.000000  102.000000
max     1.000000e+07  2.360457e+06  42.000000  3604.000000  400.000000

      seats    age
count  7906.000000  7906.000000
mean    5.416393   12.016064
std     0.959208    3.863695
min     2.000000    6.000000
25%     5.000000    9.000000
50%     5.000000   11.000000
75%     5.000000   14.000000
max     14.000000   32.000000

### DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
Index: 7906 entries, 0 to 8127
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   selling_price  7906 non-null    int64
1   km_driven     7906 non-null    int64
2   fuel         7906 non-null    object
3   seller_type   7906 non-null    object
4   transmission  7906 non-null    object
5   owner        7906 non-null    object
6   mileage       7906 non-null    float64
7   engine        7906 non-null    float64
8   max_power     7906 non-null    float64
9   seats        7906 non-null    float64
10  brand         7906 non-null    object
11  age           7906 non-null    int64
dtypes: float64(4), int64(3), object(5)
memory usage: 803.0+ KB
None

### Missing values:
selling_price  0
km_driven     0
fuel          0
seller_type   0
transmission  0
owner         0
mileage       0
engine        0
max_power     0
seats         0
brand         0
age           0
dtype: int64

```

Kode di atas merupakan perintah untuk import library yang dibutuhkan. Kemudian dilakukan load data untuk membaca dataset. Dilakukan juga cleaning untuk menghapus baris yang memiliki nilai kosong (dropna). Selanjutnya dilakukan Feature Engineering yang membuat fungsi extract_number menggunakan regex untuk mengambil angka saja dari teks spesifikasi seperti "1248 CC" menjadi 1248.0 pada kolom mileage, engine, dan max_power. Selain itu, mengekstrak kata pertama dari kolom nama menjadi brand (merek) dan menghitung umur mobil (age). Pada tahap akhir dilakukan Drop Columns untuk menghapus kolom yang kotor atau sudah tidak terpakai seperti name, year, dan torque.

Cell 2: Outlier Detection and Removal

```

import matplotlib.pyplot as plt
import seaborn as sns

# Calculate IQR for original df['selling_price']
Q1_orig = df['selling_price'].quantile(0.25)
Q3_orig = df['selling_price'].quantile(0.75)
IQR_orig = Q3_orig - Q1_orig

lower_bound_orig = Q1_orig - 1.5 * IQR_orig
upper_bound_orig = Q3_orig + 1.5 * IQR_orig

outliers_orig = df[(df['selling_price'] < lower_bound_orig) | (df['selling_price'] > upper_bound_orig)]

print(f"Q1 (original df): {Q1_orig}")
print(f"Q3 (original df): {Q3_orig}")
print(f"IQR (original df): {IQR_orig}")
print(f"Lower Bound (original df): {lower_bound_orig}")
print(f"Upper Bound (original df): {upper_bound_orig}")
print(f"Number of outliers in original `selling_price`: {len(outliers_orig)}")

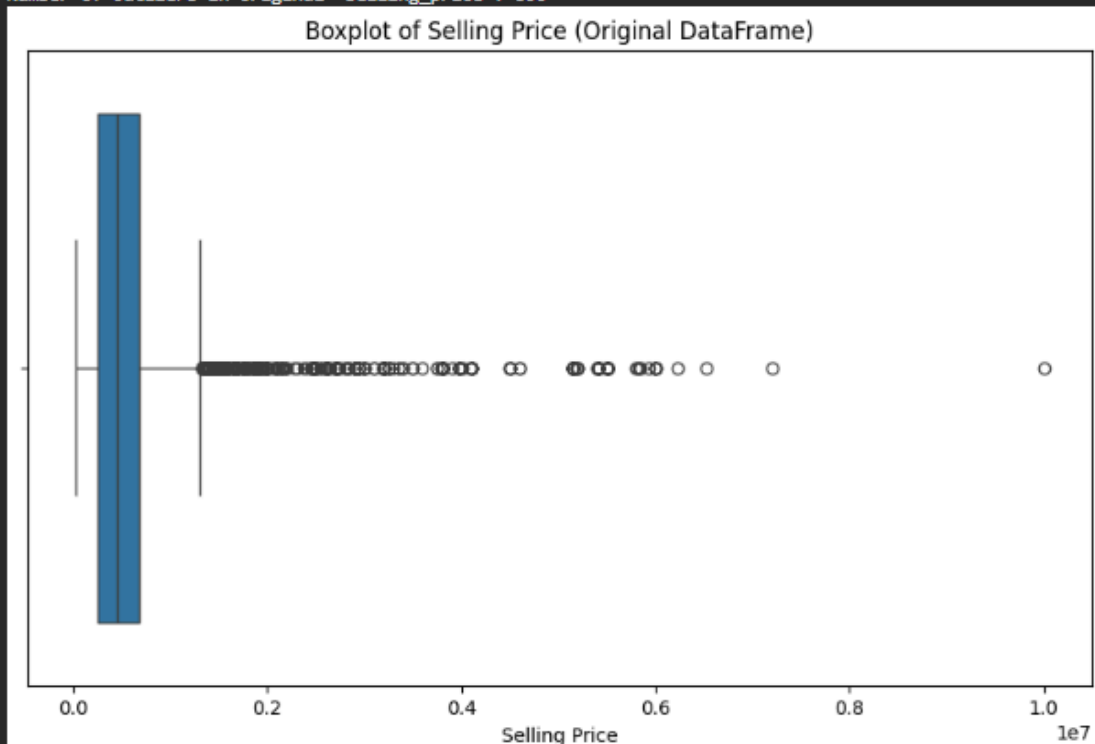
plt.figure(figsize=(10, 6))
sns.boxplot(x=df['selling_price'])
plt.title('Boxplot of Selling Price (Original DataFrame)')
plt.xlabel('Selling Price')
plt.show()

```

```

... Q1 (original df): 254999.0
Q3 (original df): 675000.0
IQR (original df): 420001.0
Lower Bound (original df): -375002.5
Upper Bound (original df): 1305001.5
Number of outliers in original `selling_price`: 600

```



Code dan output diatas bertujuan untuk mendeteksi *outlier* pada harga jual SEBELUM diproses. Hal yang dilakukan diantaranya menghitung kuartil (Q1, Q3) dan *Interquartile Range* (IQR) untuk melihat batas harga normal pada data awal. Kemudian dilakukan

Boxplot Visualization yang menampilkan grafik *Boxplot* untuk memvisualisasikan data harga ekstrem.

```
# Calculate IQR for df_processed['selling_price']
Q1 = df_processed['selling_price'].quantile(0.25)
Q3 = df_processed['selling_price'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers_selling_price = df_processed[(df_processed['selling_price'] < lower_bound) | (df_processed['selling_price'] > upper_bound)]

print(f"Q1 (df_processed): {Q1}")
print(f"Q3 (df_processed): {Q3}")
print(f"IQR (df_processed): {IQR}")
print(f"Lower Bound (df_processed): {lower_bound}")
print(f"Upper Bound (df_processed): {upper_bound}")
print(f"Number of outliers in 'selling_price' (df_processed): {len(outliers_selling_price)}")

# Remove outliers from df_processed
df_cleaned = df_processed[~((df_processed['selling_price'] < lower_bound) | (df_processed['selling_price'] > upper_bound))].copy()

print(f"\nShape of df_processed before outlier removal: {df_processed.shape}")
print(f"Shape of df_cleaned after outlier removal: {df_cleaned.shape}")

print("\n### Descriptive Statistics for Selling Price after Outlier Removal:")
print(df_cleaned['selling_price'].describe())

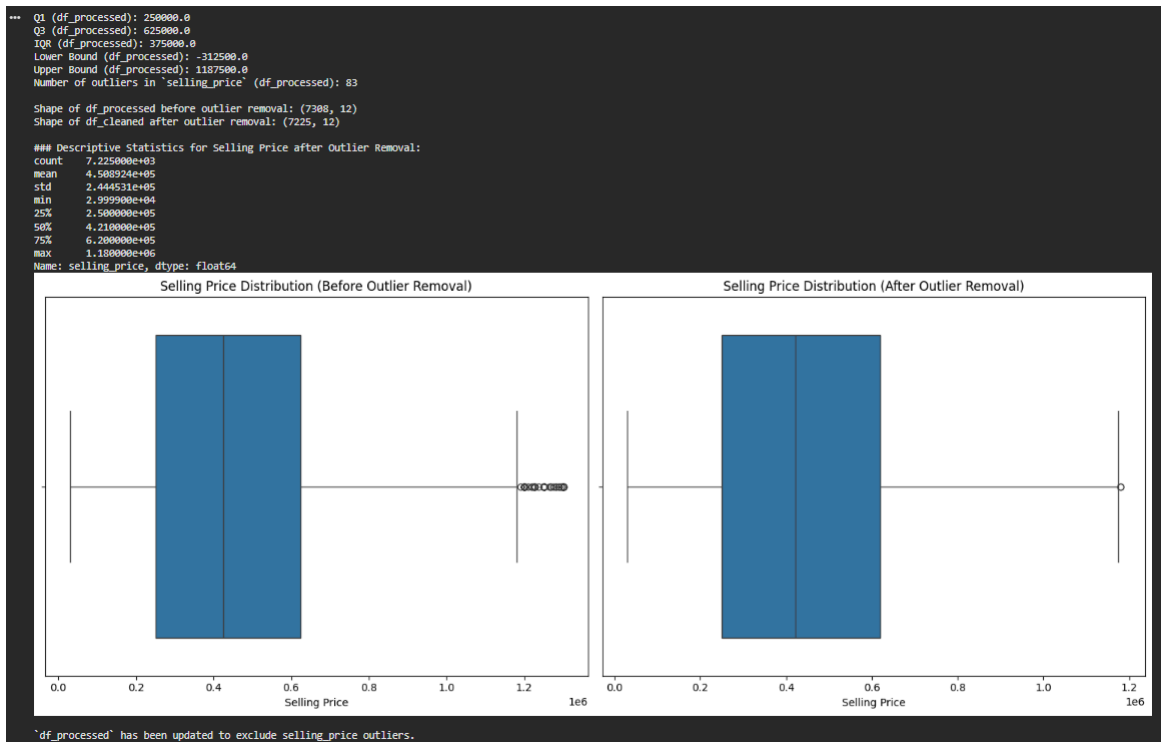
# Visualize 'selling_price' distribution before and after outlier removal
plt.figure(figsize=(15, 6))

plt.subplot(1, 2, 1)
sns.boxplot(x=df_processed['selling_price'])
plt.title('Selling Price Distribution (Before Outlier Removal)')
plt.xlabel('Selling Price')

plt.subplot(1, 2, 2)
sns.boxplot(x=df_cleaned['selling_price'])
plt.title('Selling Price Distribution (After Outlier Removal)')
plt.xlabel('Selling Price')

plt.tight_layout()
plt.show()

# Update df_processed to be df_cleaned for subsequent steps
df_processed = df_cleaned
print("\n`df_processed` has been updated to exclude selling_price outliers.")
```



Perintah diatas merupakan lanjutan sebelumnya yang bertujuan untuk mengapus outliers dengan cara menghitung ulang batas harga (IQR) pada `df_processed` dan secara aktif membuang baris data mobil yang harganya di bawah atau di atas batas kewajaran. Kemudian dilakukan Boxplot Visualization ulang yang enampilkan grafik *Boxplot* versi terbaru untuk membuktikan data sudah jauh lebih bersih.

Cell 3: Encoding Categorical Features

```

[8]
✓ Os
categorical_cols = ['brand', 'fuel', 'seller_type', 'transmission', 'owner']
label_encoders = {}

for col in categorical_cols:
    le = LabelEncoder()
    df_processed[col] = le.fit_transform(df_processed[col])
    label_encoders[col] = le # Save each encoder

# Save the dictionary of Label Encoders
joblib.dump(label_encoders, 'label_encoders.pkl')

# Sangat Penting untuk LightGBM: Ubah tipe data menjadi 'category' agar diproses otomatis oleh LGBM
for col in categorical_cols:
    df_processed[col] = df_processed[col].astype('category')

print("Categorical columns encoded and set to 'category' dtype.")
print("Dictionary of Label Encoders saved as label_encoders.pkl")

```

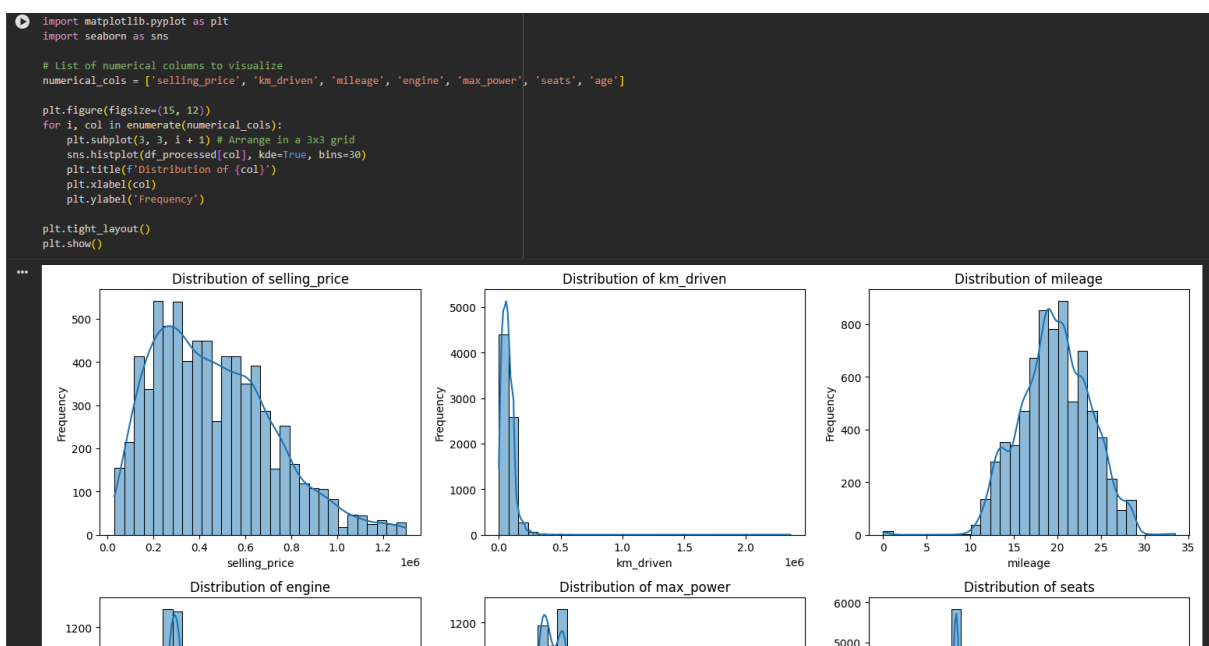
✓ Categorical columns encoded and set to 'category' dtype.
Dictionary of Label Encoders saved as label_encoders.pkl

Kode diatas merupakan perintah untuk Label Encoding yang mengubah kolom teks/kategorikal (brand, fuel, seller_type, transmission, owner) menjadi angka (0, 1, 2, dst.) menggunakan LabelEncoder dari scikit-learn.

Menyimpan Encoder: Menyimpan dictionary dari encoder ini menggunakan joblib.dump agar nanti bisa digunakan saat deployment atau testing data baru.

Casting Tipe Data: Mengubah tipe data kolom kategorikal di atas menjadi 'category' secara eksplisit agar algoritma LightGBM bisa mendeteksi dan memprosesnya secara optimal tanpa perlu One-Hot Encoding.

Cell 4: Data Distribution Visualization



Perintah diatas bertujuan untuk Plotting Distributions dengan cara membuat barisan grafik (menggunakan subplot) yang menampilkan Histogram & KDE Plot untuk seluruh kolom numerik (selling_price, km_driven, mileage, engine, max_power, seats, age) guna memahami bagaimana bentuk distribusi setiap datanya.

Cell 5: Pembagian Dataset

```
# 4. SPLIT DATA & TARGET LOG TRANSFORMATION
X = df_processed.drop('selling_price', axis=1)
y = df_processed['selling_price']

# Gunakan log1p untuk stabilitas komputasi
y_log = np.log1p(y)

X_train, X_test, y_train_log, y_test_log = train_test_split(X, y_log, test_size=0.2, random_state=42)

# Simpan harga asli dari data test untuk perhitungan metrik evaluasi nanti
y_test_original = np.exp1p(y_test_log)

print("Data siap dilatih!")
print(f"Fitur yang digunakan: {list(X.columns)}")
```

... Data siap dilatih!
Fitur yang digunakan: ['km_driven', 'fuel', 'seller_type', 'transmission', 'owner', 'mileage', 'engine', 'max_power', 'seats', 'brand', 'age']

Kode di atas merupakan perintah untuk memisahkan kolom input atau prediktor (X) dengan kolom target yang akan ditebak (y). Kemudian dilakukan Log Transformation yang mengubah target selling_price menggunakan logaritma natural (np.log1p) untuk menstabilkan prediksi dan meredam efek data yang tidak merata. Selanjutnya dilakukan Train-Test Split yang memecah data untuk dihafal oleh model (80% Train) dan untuk diuji (20% Test). Langkah akhir berupa Inverse Target yang menyimpan data target uji dalam bentuk skala harga aslinya (y_test_original) menggunakan fungsi eksponensial np.expml.

Cell 6: Pelatihan Model XGBoost dan LightGBM

```
import xgboost as xgb
import lightgbm as lgb

# --- XGBoost Model Training ---
print("Training XGBoost Regressor...")
xgb_regressor = xgb.XGBRegressor(
    objective='reg:squarederror',
    n_estimators=100,
    learning_rate=0.1,
    max_depth=5,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    enable_categorical=True # Added to handle 'category' dtype columns
)
# Use X_train and y_train_log for training
xgb_regressor.fit(X_train, y_train_log)

# Make predictions on the test set (log-transformed)
y_pred_xgb_log = xgb_regressor.predict(X_test)

# Inverse transform the predictions to the original scale
y_pred_xgb = np.expml(y_pred_xgb_log)
print("XGBoost model trained and predictions made.")

# --- LightGBM Model Training ---
print("\nTraining LightGBM Regressor...")
lgbm_regressor = lgb.LGBMRegressor(
    objective='regression',
    n_estimators=100,
    learning_rate=0.1,
    num_leaves=31,
    max_depth=-1,
    random_state=42
)
# Use X_train and y_train_log for training
lgbm_regressor.fit(X_train, y_train_log)

# Make predictions on the test set (log-transformed)
y_pred_lgbm_log = lgbm_regressor.predict(X_test)

# Inverse transform the predictions to the original scale
y_pred_lgbm = np.expml(y_pred_lgbm_log)
print("LightGBM model trained and predictions made.")
```

```

** Training XGBoost Regressor...
XGBoost model trained and predictions made.

Training LightGBM Regressor...
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.001066 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 839
[LightGBM] [Info] Number of data points in the train set: 5846, number of used features: 11
[LightGBM] [Info] Start training from score 12.855126
LightGBM model trained and predictions made.

```

Kode di atas merupakan perintah untuk Train XGBoost yang mendefinisikan model XGBRegressor (diatur agar mendukung kolom kategorikal) dan melatihnya dengan data Train. Selain itu digunakan model lain yaitu Train LightGBM yang mendefinisikan model LGBMRegressor dan melatihnya dengan data yang sama. Langkah selanjutnya adalah melakukan Predict & Inverse yang menyuruh kedua model menebak data Test, lalu hasil tebakannya dikembalikan dari bentuk logaritma menjadi nominal harga asli (np.expml).

Cell 7: Evaluasi Model dengan MAE, MAPE, RMSE, MSE, dan R2

```

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

def evaluate_model(y_true, y_pred, model_name="Model"):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_pred)

    # Calculate MAPE, handling division by zero and infinity
    errors = np.abs((y_true - y_pred) / y_true)
    # Replace inf with nan, then nan with 0 or exclude them
    errors[np.isinf(errors)] = np.nan # Treat infinite values as NaN
    errors = errors[~np.isnan(errors)] # Remove NaN values from calculation

    if len(errors) > 0:
        mape = np.mean(errors) * 100
    else:
        mape = 0.0 # If all values were NaN or inf, MAPE is 0

    print(f"### {model_name} Evaluation Metrics:")
    print(f"MAE (Mean Absolute Error) : {mae:.2f}")
    print(f"MAPE (Mean Absolute Percentage Error) : {mape:.2f}%")
    print(f"RMSE (Root Mean Squared Error) : {rmse:.2f}")
    print(f"MSE (Mean Squared Error) : {mse:.2f}")
    print(f"R2 Score : {r2:.2f}")
    print("\n")

    # Evaluate XGBoost Model using y_test_original
    evaluate_model(y_test_original, y_pred_xgb, "XGBoost Regressor")

    # Evaluate LightGBM Model using y_test_original
    evaluate_model(y_test_original, y_pred_lgbm, "LightGBM Regressor")

```

```

...   ### XGBoost Regressor Evaluation Metrics:
      MAE (Mean Absolute Error)           : 57538.78
      MAPE (Mean Absolute Percentage Error) : 15.02%
      RMSE (Root Mean Squared Error)       : 80738.69
      MSE (Mean Squared Error)            : 6518736571.66
      R2 Score                            : 0.90

      ### LightGBM Regressor Evaluation Metrics:
      MAE (Mean Absolute Error)           : 56032.66
      MAPE (Mean Absolute Percentage Error) : 14.76%
      RMSE (Root Mean Squared Error)       : 79690.94
      MSE (Mean Squared Error)            : 6350646482.47
      R2 Score                            : 0.90

```

Kode di atas merupakan perintah untuk Define Metric Function yang membuat fungsi buatan `evaluate_model` untuk merangkum perhitungan metrik kesalahan prediksi (MAE, MAPE, RMSE, MSE, R2). Di dalamnya terdapat pengaman untuk menghindari error jika terdapat pembagian nol. Kemudian dilakukan Evaluate Models yang memanggil fungsi tersebut agar langsung memunculkan rapor nilai untuk XGBoost dan LightGBM. Untuk tahap selanjutnya akan dipilih model LightBM untuk implementasi UI/UX karena akurasiya sedikit lebih baik.

Cell 8: Visualisasi Perbandingan Model Menggunakan ScatterPlot

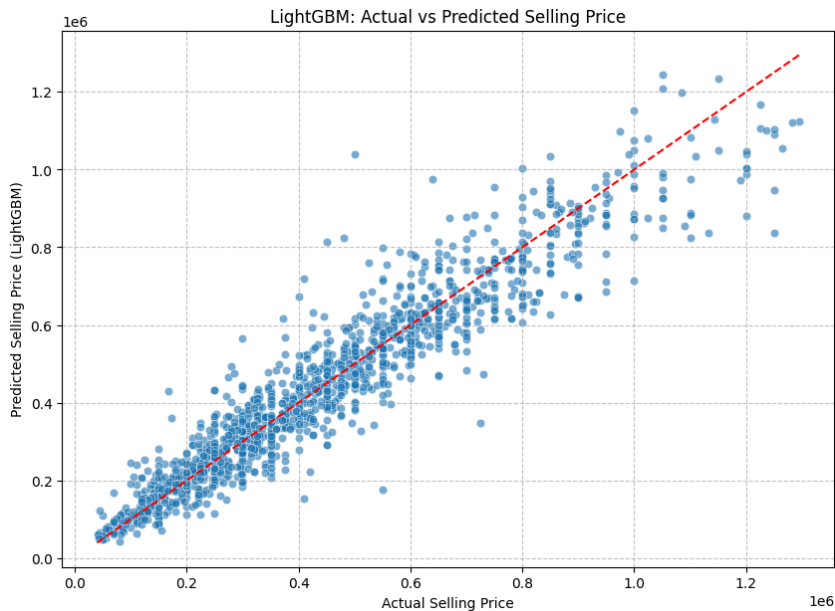
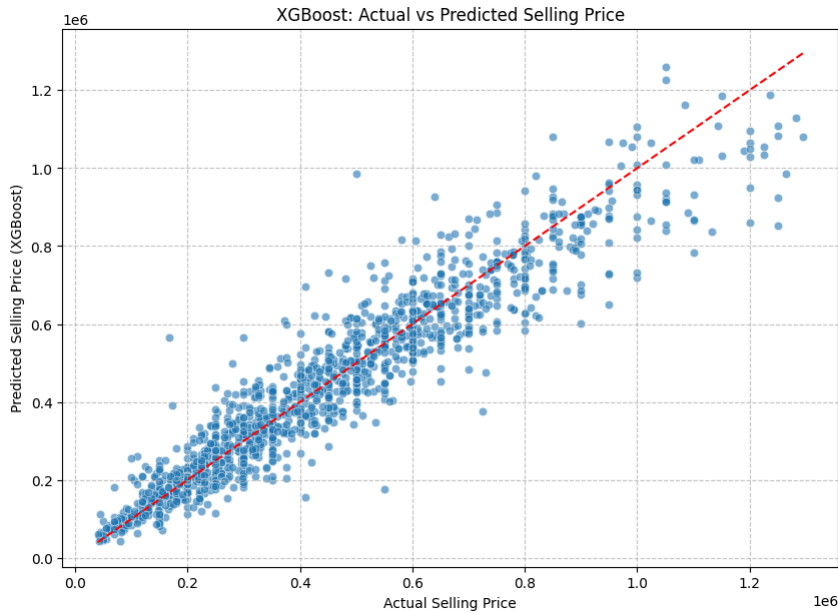
```

import matplotlib.pyplot as plt
import seaborn as sns

# Create a scatter plot for Actual vs Predicted Price (XGBoost)
plt.figure(figsize=(10, 7))
sns.scatterplot(x=y_test_original, y=y_pred_xgb, alpha=0.6)
plt.plot([y_test_original.min(), y_test_original.max()], [y_test_original.min(), y_test_original.max()], 'r--')
plt.title('XGBoost: Actual vs Predicted Selling Price')
plt.xlabel('Actual Selling Price')
plt.ylabel('Predicted Selling Price (XGBoost)')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

# Create a scatter plot for Actual vs Predicted Price (LightGBM)
plt.figure(figsize=(10, 7))
sns.scatterplot(x=y_test_original, y=y_pred_lgbm, alpha=0.6)
plt.plot([y_test_original.min(), y_test_original.max()], [y_test_original.min(), y_test_original.max()], 'r--')
plt.title('LightGBM: Actual vs Predicted Selling Price')
plt.xlabel('Actual Selling Price')
plt.ylabel('Predicted Selling Price (LightGBM)')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

```



Langkah akhirnya adalah membuat Scatter Plot Prediction yang menampilkan dua buah grafik sebaran titik (Scatter Plot) yang membandingkan Harga Asli vs Harga Tebakan (untuk masing-masing model). Grafik ini dilengkapi dengan garis putus-putus merah yang merepresentasikan titik "tebakan sempurna".

4. Implementasi UI/UX Menggunakan Streamlit

Berikut adalah penjabaran kode dan antarmuka aplikasi Streamlit untuk sistem deteksi dini penyakit diabetes, yang dibagi menjadi file utilitas dan file aplikasi utama.

1. File app.py (Aplikasi Utama Streamlit)

Konfigurasi Halaman Dasar (Bagian 1)

```

import streamlit as st
import pandas as pd
import numpy as np
import joblib
from datetime import datetime

# =====
# 1. KONFIGURASI HALAMAN
# =====
st.set_page_config(
    page_title="Kalkulator Harga Mobil Bekas",
    page_icon="8308414.png",
    layout="centered"
)

```

Tujuan UX diatas untuk memberikan identitas visual awal pada *tab browser* dengan mengimplementasikan `page_title`, `page_icon`, dan `layout="centered"` agar tata letak halaman utama berada di tengah layar

Manajemen Aset dan Penanganan Error (Bagian 2)

```

# =====
# 2. MEMUAT MODEL DAN ENCODER
# =====
# Menggunakan @st.cache_resource agar model tidak di-load ulang
# setiap kali ada interaksi
@st.cache_resource
def load_assets():
    xgb_model = joblib.load('xgboost_model.pkl')
    lgbm_model = joblib.load('lightgbm_model.pkl')
    encoders = joblib.load('label_encoders.pkl')
    return xgb_model, lgbm_model, encoders

try:
    xgb_model, lgbm_model, encoders = load_assets()
except FileNotFoundError:
    st.error("⚠ File model (.pkl) tidak ditemukan. Pastikan
'xgboost_model.pkl', 'lightgbm_model.pkl', dan
'label_encoders.pkl' ada di folder yang sama dengan app.py.")
    st.stop()

```

Tujuan UX diatas untuk memastikan aplikasi berjalan responsif dan menangani *error* dengan cara yang ramah pengguna. Implementasinya berupa penggunaan dekorator `@st.cache_resource` pada fungsi `load_assets()` untuk mencegah aplikasi memuat ulang model *Machine Learning* yang berat setiap kali pengguna mengganti nilai input. Blok `try...except` menangani kemungkinan *file* model hilang, `st.error()` akan menampilkan peringatan visual yang jelas dan menghentikan eksekusi menggunakan `st.stop()`.

Struktur Antarmuka dan Input Pengguna (Bagian 3)

```
# =====
# 3. ANTARMUKA PENGGUNA (UI)
# =====
st.title("AI Prediksi Harga Mobil Bekas")
st.markdown("""
Aplikasi ini menggunakan Kecerdasan Buatan (AI) untuk memprediksi
harga jual mobil bekas berdasarkan spesifikasinya.
Silakan masukkan detail kendaraan di bawah ini.
""")

st.divider()

# Membagi layar menjadi 2 kolom untuk tampilan yang lebih rapi
col1, col2 = st.columns(2)

with col1:
    st.subheader("Informasi Dasar")
    # Mengambil daftar pilihan dari Label Encoder agar sesuai
    dengan data training
    brand = st.selectbox("Merek Mobil",
encoders['brand'].classes_)
    tahun = st.number_input("Tahun Pembuatan", min_value=1990,
max_value=datetime.now().year, value=2018, step=1)
    km_driven = st.number_input("Jarak Tempuh (Kilometer)",
min_value=0, value=50000, step=1000)
    owner = st.selectbox("Kepemilikan Tangan Ke-",
encoders['owner'].classes_)
    seller_type = st.selectbox("Dijual Oleh",
encoders['seller_type'].classes_)

with col2:
    st.subheader("Spesifikasi Mesin")
```

```

        fuel = st.selectbox("Jenis Bahan Bakar",
encoders['fuel'].classes_)
        transmission = st.selectbox("Transmisi",
encoders['transmission'].classes_)
        mileage = st.number_input("Konsumsi BBM (km/liter)",
min_value=0.0, value=18.0, step=0.1)
        engine = st.number_input("Kapasitas Mesin (CC)",
min_value=500, max_value=8000, value=1200, step=100)
        max_power = st.number_input("Tenaga Maksimal (BHP)",
min_value=10.0, max_value=1000.0, value=80.0, step=1.0)
        seats = st.number_input("Jumlah Kursi", min_value=2,
max_value=14, value=5, step=1)

st.divider()

# Pilihan Model AI
st.subheader("Pengaturan AI")
model_choice = st.radio("Pilih Mesin Prediksi (Model AI):",
["LightGBM (Disarankan)", "XGBoost"], horizontal=True)

# Tombol Prediksi
if st.button("Hitung Prediksi Harga", use_container_width=True,
type="primary"):

```

Tujuan UX diatas untuk menyediakan formulir input yang terorganisir, mudah diisi, dan meminimalkan kesalahan input dengan mengimplementasikan Header: st.title dan st.markdown yang memberikan konteks tentang tujuan aplikasi. Selain itu ada Tata Letak Kolom dengan penggunaan st.columns(2) yang membagi formulir menjadi dua area: "Informasi Dasar" dan "Spesifikasi Mesin". Selain itu ada Input Terkontrol berupa st.selectbox yang digunakan untuk data kategorikal (merek, bahan bakar, dsb.) dan st.number_input digunakan untuk data numerik dengan batasan min_value, max_value, dan step yang rasional.

Tujuan UX lainnya yaitu memberikan kontrol kepada pengguna dan memicu aksi. Implementasinya berupa pengaturan AI menggunakan st.radio secara horizontal untuk memilih algoritma prediksi. Menambahkan label "(Disarankan)" pada LightGBM membantu membimbing pengguna awam untuk membuat keputusan yang optimal.

Pemrosesan Data di Latar Belakang (Bagian 4 & 5)

```

# =====
# 4. PEMROSESAN DATA (PREPROCESSING)
# =====
# Menghitung umur mobil sesuai logika di notebook:
datetime.now().year - year
current_year = datetime.now().year
age = current_year - tahun

# Menyusun data ke dalam DataFrame
input_data = pd.DataFrame({
    'km_driven': [km_driven],
    'fuel': [fuel],
    'seller_type': [seller_type],
    'transmission': [transmission],
    'owner': [owner],
    'mileage': [mileage],
    'engine': [engine],
    'max_power': [max_power],
    'seats': [seats],
    'brand': [brand],
    'age': [age]
})

# Mengubah data teks menjadi angka (Label Encoding) seperti
saat training
categorical_cols = ['brand', 'fuel', 'seller_type',
'transmission', 'owner']
for col in categorical_cols:
    # Menggunakan transform dari encoder yang sudah dilatih
    input_data[col] = encoders[col].transform(input_data[col])
    # Mengubah ke tipe 'category' karena LightGBM sangat
    bergantung pada format ini
    input_data[col] = input_data[col].astype('category')

# Memastikan urutan kolom sama persis dengan X_train di
notebook
expected_columns = ['km_driven', 'fuel', 'seller_type',
'transmission', 'owner', 'mileage', 'engine', 'max_power',
'seats', 'brand', 'age']
input_data = input_data[expected_columns]

# =====
# 5. PREDIKSI

```



```
# =====
with st.spinner('AI sedang menganalisis harga pasar...'):
    if "LightGBM" in model_choice:
        pred_log = lgbm_model.predict(input_data)
    else:
        # XGBoost di notebook Anda mungkin membutuhkan format
        khusus jika kategori diaktifkan
        pred_log = xgb_model.predict(input_data)

    # Karena di notebook target di-log menggunakan np.log1p,
    kita kembalikan menggunakan np.expm1
    pred_price = np.expm1(pred_log)[0]
```

Tujuan UX diatas untuk memberikan umpan balik visual (*feedback*) saat aplikasi sedang bekerja. Implementasinya berupa penggunaan konversi input pengguna ke dalam format DataFrame dan memanggil model, with st.spinner(...) digunakan. Ini akan menampilkan animasi *loading*.

6. Menampilkan Hasil Visual (Bagian 6)

```
# =====

# 6. MENAMPILKAN HASIL

# =====

st.success("Prediksi Selesai!")

# Menampilkan kartu hasil prediksi dengan format mata uang

st.markdown(f"""

    <div style="padding:20px; border-radius:10px;
text-align:center;">

        <h3 style="margin-bottom:0;">Estimasi Harga Jual:</h3>

        <h1 style="color:#2e7bcf; margin-top:5px;">USD
{pred_price:,.0f}</h1>

        <p style="font-size:14px; "><i>*Harga dapat bervariasi
tergantung kondisi fisik dan pajak kendaraan.</i></p>

    </div>

""")
```

```
</div>

"", unsafe_allow_html=True)
```

Tujuan UX diatas untuk menampilkan jawaban akhir dengan jelas dan meyakinkan. Implementasinya berupa `st.success("Prediksi Selesai!")` yang memberikan konfirmasi bahwa proses berhasil. Hasil prediksi menggunakan HTML dan CSS sebaris dalam `st.markdown` agar "kartu hasil" bergaya *dashboard* bisa dihasilkan (`text-align:center`, (`<h1>`) dengan warna khusus, formatting koma untuk memisahkan ribuan (`USD pred_price:,.0f`)). Catatan penafian (*disclaimer*) kecil di bagian bawah ("**Harga dapat bervariasi...*") mengelola ekspektasi pengguna bahwa ini adalah sekadar estimasi model.

➤ Tampilan Antarmuka Web

AI Prediksi Harga Mobil Bekas

Aplikasi ini menggunakan Kecerdasan Buatan (AI) untuk memprediksi harga jual mobil bekas berdasarkan spesifikasinya. Silakan masukkan detail kendaraan di bawah ini.

Informasi Dasar	Spesifikasi Mesin
Merek Mobil Ambassador	Jenis Bahan Bakar CNG
Tahun Pembuatan 2018	Transmisi Automatic
Jarak Tempuh (Kilometer) 50000	Konsumsi BBM (km/liter) 18.00
Kepemilikan Tangan Ke- First Owner	Kapasitas Mesin (CC) 1200
Dijual Oleh Dealer	Tenaga Maksimal (BHP) 80.00

